Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
ISIS4826 – Robótica Móvil

Proyecto #2
Planificación Computacional de Caminos Geométricos, Ejecución (Modo Autónomo) y Relocalización

---

2026-1

Profesor: Fernando De la Rosa

28 de abril de 2026

Índice

1. Método de Planificación **2**
1.1. Descripción del método computacional . . . 2
1.1.1. Construcción del C-espacio . . . 2
1.1.2. Algoritmo A* . . . 2
1.1.3. Generación del camino geométrico . . . 2
1.2. Resultado del método de planificación . . . 3
1.3. Traducción del camino en comandos al robot . . . 3
1.4. Tabla comparativa q_f vs. $q_{f\text{-est}}$ . . . 4
1.5. Posibles mejoras del método de planificación . . . 4

2. Método de Relocalización **4**
2.1. Descripción del método . . . 4
2.2. Cálculo de q_{act} . . . 5
2.3. Tabla comparativa $q_{f\text{-est}}$ vs. q_{act} . . . 6
2.4. Causas de error en la localización . . . 6
2.5. Análisis de resultados en simulación . . . 6

3. Videos ilustrativos **7**

A. Estructura del proyecto ROS2 **7**

1. Método de Planificación

1.1. Descripción del método computacional

El método de planificación implementado es **A* sobre una cuadrícula del espacio de configuraciones (C-espacio)**. La estrategia combina la construcción de C-obstáculos del proyecto 1 con un algoritmo de búsqueda heurística que garantiza optimalidad respecto al costo definido.

1.1.1. Construcción del C-espacio

El espacio de trabajo se discretiza en una cuadrícula de celdas cuadradas de $\Delta = 0{,}25$ m de lado. El robot se modela como un cuadrado de 0,30 m de lado (radio $r = 0{,}15$ m), pero se usa un radio conservador de $r = 0{,}20$ m para garantizar márgenes de seguridad en ejecución real.

Cada celda se clasifica en tres estados:

- **LIBRE (0):** el robot puede ocuparla sin riesgo de colisión.
- **SEMI-LIBRE (1):** el centro de la celda está a menos de $r+0{,}10$ m de un obstáculo. Se penaliza con un factor $\times 5$ en A* para que el robot se aleje de paredes y obstáculos.
- **OCUPADA (2):** el robot colisionaría. A* no la explora.

Los C-obstáculos se calculan expandiendo cada obstáculo rectangular en $r = 0{,}20$ m en todas las direcciones. Las paredes del escenario reciben el mismo tratamiento: toda celda cuyo centro esté a menos de r m de un borde queda marcada como OCUPADA.

1.1.2. Algoritmo A*

La búsqueda se realiza con conectividad de 8 vecinos (movimientos diagonales permitidos). La función de evaluación es:

$$f(n) = g(n) + h(n) \tag{1}$$

donde $g(n)$ es el costo acumulado desde el inicio (distancia euclídea en celdas, con penalización $\times 5$ para celdas SEMI-LIBRE) y $h(n)$ es la heurística admisible de distancia euclídea a la celda objetivo.

1.1.3. Generación del camino geométrico

Una vez obtenida la secuencia de celdas se aplican dos postprocesos:

1. **Simplificación:** se eliminan los puntos colineales consecutivos (cambios de dirección menores a 0,01 rad no generan waypoint intermedio).
2. **Segmentación:** los segmentos rectos de más de 4 celdas ($\approx 1{,}0$ m) se subdividen para reducir la deriva odométrica acumulada en traslaciones largas.

El camino final tiene la forma:

$$\langle q_0,\ q_{\text{rot}_{0\text{-}1}},\ q_1,\ q_{\text{rot}_{1\text{-}2}},\ q_2,\ \ldots,\ q_n,\ q_{\text{rot}_n}\rangle \tag{2}$$

Cada $q_i = (x_i,\ y_i,\ \theta_i)$ y cada $q_{\text{rot}_{i\text{-}i+1}}$ comparte posición con q_i pero tiene el ángulo necesario para apuntar hacia q_{i+1}.

1.2. Resultado del método de planificación

La Tabla 1 resume los caminos obtenidos para las 6 escenas. La última posición del camino es *exactamente* (x_f, y_f) del problema (sin redondeo a celda) y la orientación final es exactamente θ_f.

Cuadro 1: Resumen de planificación A* para las 6 escenas.

Escena	$q_0 = (x, y, \theta)$	$q_f = (x, y, \theta)$	Obstáculos	Waypoints	Long. camino (m)
1	(0,75, 0,75, 0)	(3,25, 4,25, 90)	4	20	4,54
2	(0,75, 0,75, 0)	(3,25, 4,25, 90)	5	14	4,54
3	(0,75, 0,75, 0)	(3,25, 4,25, 90)	8	18	4,83
4	(0,75, 0,75, 0)	(3,25, 4,25, 90)	8	22	5,87
5	(3,25, 0,75, 180)	(3,25, 4,25, 90)	8	24	4,84
6	(3,25, 0,75, 180)	(3,25, 4,25, 90)	10	26	6,13

El camino solución se guarda en un archivo `.txt` con una configuración (x, y, θ) por línea. Al finalizar la ejecución se añaden las configuraciones $q_{f\text{-est}}$ y q_{act} como comentarios:

```
0.7500,0.7500,0.0000
...
3.2500,4.2500,90.0000
# qf-est: 3.XXXX,4.XXXX,XX.XXXX
# qact:   3.XXXX,4.XXXX,XX.XXXX
```

1.3. Traducción del camino en comandos al robot

El módulo `PathExecutor` procesa los waypoints a 10 Hz mediante una máquina de estados de dos fases por waypoint:

1. **ROTATE:** el robot gira en su sitio hasta $|\Delta\theta| < 0{,}05$ rad. Velocidad angular proporcional: $\omega = \text{clip}(2{,}0\,\Delta\theta,\ -0{,}5,\ 0{,}5)$ rad/s.
2. **TRANSLATE:** el robot avanza hacia (x_i, y_i) con velocidad $v = \text{clip}(0{,}4\,d,\ 0{,}06,\ 0{,}20)$ m/s. Una corrección de heading suave actúa solo cuando el error angular supera 8.

El robot se detiene completamente entre fases para evitar oscilaciones en los giros. La tolerancia de llegada es TOL_DIST = 0,20 m para traslación y TOL_ANGLE = 0,05 rad para rotación.

1.4. Tabla comparativa q_f vs. $q_{f\text{-est}}$

Nota: q_f es idéntico en las 6 escenas — (3,25, 4,25, 90) — porque el enunciado así lo define. $q_{f\text{-est}}$ es la posición reportada por la odometría de Gazebo al finalizar cada ejecución.

Cuadro 2: Configuración teórica q_f vs. estimada $q_{f\text{-est}}$ (de odometría Gazebo).

	$q_f = (x,\ y,\ \theta)$	$q_{f\text{-est}} = (x,\ y,\ \theta)$	Δdist (m)	$\Delta\theta$ (°)
Escena 1	(3,25, 4,25, 90)	(3,110, 4,208, 88,5)	0,1463	1,47
Escena 2	(3,25, 4,25, 90)	(2,613, 3,321, 80,9)	1,1267	9,12
Escena 3	(3,25, 4,25, 90)	(2,684, 3,230, 81,6)	1,1667	8,40
Escena 4	(3,25, 4,25, 90)	(3,060, 4,296, 90,3)	0,1954	−0,29
Escena 5	(3,25, 4,25, 90)	(3,085, 4,173, 87,2)	0,1821	2,80
Escena 6	(3,25, 4,25, 90)	(2,953, 4,089, 85,8)	0,3378	4,20

1.5. Posibles mejoras del método de planificación

- **Suavizado de caminos (Bézier / B-spline):** el camino actual produce movimientos tipo "girar–avanzar–girar" que acumulan error odométrico. Un camino suavizado reduciría las rotaciones intermedias.
- **RRT* / PRM:** para escenarios con pasillos angostos, los métodos de muestreo aleatorio encuentran caminos factibles más rápido que A* en cuadrículas densas.
- **Resolución adaptativa:** usar celdas más pequeñas cerca de los obstáculos y más grandes en espacios abiertos mejoraría la calidad del camino sin incrementar excesivamente el tiempo de cómputo.
- **Corrección odométrica en línea:** integrar lecturas LiDAR durante la ejecución (no solo al final) permitiría corregir desviaciones acumuladas en trayectos largos, especialmente útil para las escenas 4 y 6 que tienen caminos de más de 5 m.

2. Método de Relocalización

2.1. Descripción del método

Al llegar al destino q_f, el robot se detiene durante 3 s para estabilizar las lecturas LiDAR. Luego mide las distancias reales al obstáculo frontal ($d^{\text{med}}_{\text{frente}}$) y al obstáculo derecho ($d^{\text{med}}_{\text{derecha}}$).

Para las 6 escenas: $q_f = (3{,}25,\ 4{,}25,\ 90)$, $d^{\text{teo}}_{\text{frente}} = 0{,}8$ m, $d^{\text{teo}}_{\text{derecha}} = 0{,}78$ m. Los obstáculos de referencia (pared norte y pared/obstáculo este) se encuentran en:

$$\mathbf{p}_{\text{obs_frente}} = (3{,}25,\ 4{,}25) + 0{,}8\,(\cos 90,\ \sin 90) = (3{,}25,\ 5{,}05) \tag{3}$$

$$\mathbf{p}_{\text{obs_derecha}} = (3{,}25,\ 4{,}25) + 0{,}78\,(\sin 90,\ -\cos 90) = (4{,}03,\ 4{,}25) \tag{4}$$

2.2. Cálculo de $q_{\mathbf{act}}$

Paso 1 – Vectores de dirección según $\theta_{\mathbf{est}}$ (orientación reportada por la odometría):

$$\hat{f}_{\text{est}} = (\cos\theta_{\text{est}},\ \sin\theta_{\text{est}}), \qquad \hat{d}_{\text{est}} = (\sin\theta_{\text{est}},\ -\cos\theta_{\text{est}}) \tag{5}$$

Paso 2 – Posición estimada del robot desde cada obstáculo:

$$\mathbf{q}_{\text{act}}^{(f)} = \mathbf{p}_{\text{obs_frente}} - d_{\text{frente}}^{\text{med}}\,\hat{f}_{\text{est}} \tag{6}$$

$$\mathbf{q}_{\text{act}}^{(d)} = \mathbf{p}_{\text{obs_derecha}} - d_{\text{derecha}}^{\text{med}}\,\hat{d}_{\text{est}} \tag{7}$$

Paso 3 – Promedio:

$$\mathbf{q}_{\text{act}} = \frac{\mathbf{q}_{\text{act}}^{(f)} + \mathbf{q}_{\text{act}}^{(d)}}{2}, \qquad \theta_{\text{act}} = \theta_{\text{est}} \tag{8}$$

Verificación: Si $d_{\text{med}} = d_{\text{teo}}$ y $\theta_{\text{est}} = 90$ (ejecución perfecta), se recupera exactamente $q_{\text{act}} = q_f = (3{,}25,\ 4{,}25,\ 90)$, lo que confirma la corrección del método. La Tabla 3 muestra la sensibilidad de q_{act} al error angular de la odometría (con LiDAR perfecto):

Cuadro 3: Sensibilidad de q_{act} al error angular θ_{est} (con $d_{\text{frente}}^{\text{med}} = 0{,}8$ m y $d_{\text{derecha}}^{\text{med}} = 0{,}78$ m).

θ_{est} (°)	x_{act} (m)	y_{act} (m)	$\Delta\text{dist}(q_f, q_{\text{act}})$ (m)
85	3,2166	4,2855	0,049
87	3,2296	4,2710	0,029
88	3,2363	4,2639	0,020
89	3,2431	4,2569	0,010
90	**3,2500**	**4,2500**	**0,000**
91	3,2570	4,2433	0,010
92	3,2642	4,2366	0,020
93	3,2715	4,2301	0,029
95	3,2863	4,2175	0,049

Un error de 5 en θ_{est} produce $\approx 0{,}049$ m de error en q_{act}.

2.3. Tabla comparativa $q_{f\text{-est}}$ vs. q_{act}

Cuadro 4: Configuración odométrica $q_{f\text{-est}}$ vs. real relocalizada q_{act} (de Gazebo).

	$q_{f\text{-est}} = (x,\ y,\ \theta)$	$q_{\text{act}} = (x,\ y,\ \theta)$	Δdist (m)	$\Delta\theta$ (°)
Escena 1	(3,110, 4,208, 88,5)	(3,184, 4,259, 88,5)	0,0904	0,00
Escena 2	(2,613, 3,321, 80,9)	(3,113, 4,348, 80,9)	1,1426	0,00
Escena 3	(2,684, 3,230, 81,6)	(3,141, 4,276, 81,6)	1,1415	0,00
Escena 4	(3,060, 4,296, 90,3)	(3,172, 4,290, 90,3)	0,1119	0,00
Escena 5	(3,085, 4,173, 87,2)	(3,164, 4,247, 87,2)	0,1079	0,00
Escena 6	(2,953, 4,089, 85,8)	(3,131, 4,221, 85,8)	0,2211	0,00

2.4. Causas de error en la localización

- **Deslizamiento de ruedas:** principal fuente de error odométrico; se acumula en cada giro y traslación. Las escenas 4 y 6 (caminos de 5,87 m y 6,13 m respectivamente) son las más propensas a acumular error.
- **Resolución de la cuadrícula ($\Delta = 0{,}25$ m):** el planificador trabaja con posiciones discretizadas; el último segmento del camino sí termina en las coordenadas exactas de q_f, pero los waypoints intermedios presentan un error de hasta $\frac{\Delta}{2}\sqrt{2} \approx 0{,}18$ m respecto a una línea recta ideal.
- **Tolerancia de parada (TOL_DIST $= 0{,}20$ m):** el robot puede detenerse hasta 0,20 m antes de alcanzar el waypoint, lo que introduce error que se propaga al siguiente segmento.
- **Ruido LiDAR:** incertidumbre de $\pm$ algunos centímetros en las mediciones, introduciendo error residual en q_{act}.
- **Orientación no corregida:** la relocalización calcula x e y reales, pero $\theta_{\text{act}} = \theta_{\text{est}}$; cualquier error angular odométrico se mantiene en q_{act}.

2.5. Análisis de resultados en simulación

Los resultados muestran dos grupos claros de desempeño. Las escenas 1, 4 y 5 presentan errores de posición menores a 0,20 m y errores angulares inferiores a 3, lo que corresponde a ejecuciones exitosas donde el robot completó el camino planificado sin quedar atascado en ningún waypoint.

Las escenas 2 y 3, en cambio, reportan errores de $\approx 1{,}13$ m y $\approx 1{,}17$ m respectivamente. En estos casos la odometría indica que el robot se detuvo con ≈ 1 m de distancia al objetivo y un error angular de ≈ 9, evidencia de que el robot no completó el camino completo — posiblemente por quedar bloqueado en un waypoint intermedio.

Respecto a la relocalización, la diferencia $\Delta(q_{f\text{-est}}, q_{\text{act}})$ siempre tiene $\Delta\theta = 0{,}00$ ya que el método no corrige la orientación. En los casos exitosos (E1, E4, E5) la corrección de posición es de 0,09–0,11 m, lo que demuestra que el LiDAR logra mejorar la estimación de posición incluso cuando la odometría es buena. En los casos con grandes errores (E2, E3)

la corrección supera 1,14 m, acercando q_{act} al entorno de q_f real a pesar del gran desvío odométrico.

3. Videos ilustrativos

Los videos de ejecución completa en Gazebo (planificación, navegación autónoma y reporte final de q_f, $q_{f\text{-est}}$ y q_{act}) están disponibles en el siguiente enlace:

https://uniandes-my.sharepoint.com/:f:/g/personal/c_tellezs_uniandes_edu_co/IgBvLipX9PsFSbUr6vlQDs1xAVdNEaoArX3w30bY-Mp_fIg?e=pTLzfk

A. Estructura del proyecto ROS2

```
proyecto-3/
 data/
    Escena-Problema1.txt  ..  Escena-Problema6.txt
 proyecto/
    logic/
       planner.py          # A* + construcción C-espacio
       execution.py        # PathExecutor (rotate-translate)
       relocalization.py   # calcular_qact con LiDAR
       scene_parser.py     # parsear escenas y caminos .txt
       movement.py         # primitivas de movimiento relativo
       lidar.py            # wrappers LiDAR
    navigation_node.py      # Nodo ROS2 principal
 setup.py
 package.xml
```

Para iniciar el modo autónomo en la escena N desde la consola:

```
ros2 run proyecto navigation_node
# Seleccionar opción 7 → ingresar número de escena (1-6)
```

Al terminar, el nodo imprime en consola:

```
  RESULTADO FINAL

  qf      (teórico) : x=3.250  y=4.250  =90.0°
  qf-est  (odom)    : x=X.XXX  y=X.XXX  =XX.X°
  qact    (LiDAR)   : x=X.XXX  y=X.XXX  =XX.X°

  (qf - qf-est):   dist=X.XXXX m   =X.XX°
  (qf-est - qact): dist=X.XXXX m   =X.XX°
```